

重启后电机继续运行导致丢步问题分析与解决方案

整理日期：2026-06-02

1. 结论

当前代码在 CPU2 复位或整机控制板重启时，不能保证第一时间让 TMC5130 停止旧运动。若电机驱动 24V 和 TMC5130 逻辑电源没有随 CPU2 一起掉电，TMC5130 可能继续执行复位前已经写入的 `RAMPMODE`、`VMAX`、`XTARGET`。CPU2 启动后又在 GPIO 初始化阶段把 `DRV_ENN` 拉低使能驱动，随后 `App_Init()` 还会先延时 1000ms，再启动编码器采集和电机初始化。

因此故障窗口是：

- 1. 运行中 CPU2 复位，TMC5130 仍保留旧目标并继续产生运动。
- 2. CPU2 重新上电初始化 GPIO 时，`DRV_ENN` 被设置为低电平，驱动输出处于使能状态。
- 3. `App_Init()` 固定延时 1000ms，编码器 TIM1 采集尚未启动，运动变化没有进入 `g_encoder_count`。
- 4. 编码器启动后第一帧只能和 FRAM 里保存的旧 `prev_angle` 做差，如果盲区内运动超过半圈或跨圈次数不可判断，就会把真实位移折叠成错误增量，形成丢步/位置漂移。
- 5. 后续 `MotorCtrl_Init()` 才会重新下发 TMC5130 基础配置和持久化位置，此时已经无法补回盲区内的编码器位移。

当前工作区已有的 `motion_command_active` / `motion_wait_active` 改动能减少“没有软件运动命令时仍显示上/下行”的误显示，但它是 RAM 中的软件状态，CPU2 重启后会清零，无法阻止 TMC5130 在 MCU 启动盲区继续旧运动，所以不能作为本问题的根因修复。

2. 代码证据

位置	现有行为	风险
LTD_MAIN_CPU2/Core/Src/main.c:95-116	外设初始化完成后才调用 <code>App_Init()</code> 。MX_SPI2_Init() 在前，具备早期写 TMC5130 的条件。	当前没有利用 SPI2 初始化后的早期窗口先清旧运动。
LTD_MAIN_CPU2/Core/Src/gpio.c:67-70	MOTOR_SPI2_CS 和 DRV_ENN 初始均写低；底层 <code>stpr_enableDriver()</code> 也是把 ENN 写低。	GPIO 初始化阶段会使能驱动输出，且片选默认不是空闲高电平。
LTD_MAIN_CPU2/Application/Src/app_main.c:94-103	<code>App_Init()</code> 先 <code>HAL_Delay(1000)</code> ，再 <code>init_device_params()</code> 、 <code>Initialize_Encoder()</code> ，最后才 <code>MotorCtrl_Init()</code> 。	至少 1 秒内编码器未采集，电机旧运动未被强停。
LTD_MAIN_CPU2/Services/Encoder/encoder.c:108-122	上电时从 FRAM 恢复 <code>g_encoder_count/prev_angle</code> ，更新位置后只启动 TIM1 采集。	没有读取当前真实角度，也没有等待第一帧有效编码器数据。
LTD_MAIN_CPU2/Drivers/Peripherals/src/AS5145.c:269-317	TIM1 到期后才发起 SSI DMA 读取，DMA 回调成功后才 <code>Update_Encoder_Count()</code> 。	<code>Start_Encoder_Collection_TIM()</code> 返回成功只表示定时器启动，不表示编码器已经有有效首帧。
LTD_MAIN_CPU2/Services/Encoder/encoder.c:126-143	单次角度差按 <code>MAX_ANGLE / 2</code> 处理跨零点。	启动盲区内如果运动超过半圈，无法从旧角度和新角度推回真实圈数。
LTD_MAIN_CPU2/Drivers/Peripherals/src/TMC5130.c:499-513	<code>stpr_stop()</code> 会写 <code>VMAX=0</code> 、读 <code>XACTUAL</code> 、写 <code>XTARGET=XACTUAL</code> 、切回位置模式。	这是已有的安全停机能力，但启动流程没有足够早调用。
LTD_MAIN_CPU2/Services/MotorControl/motor_ctrl_driver_param.c:225-310	<code>MotorCtrl_Init()</code> 会初始化 TMC5130、恢复持久化寄存器、再写当前 <code>VMAX</code> 。	初始化发生太晚；如果旧运动已发生，恢复寄存器只会覆盖驱动坐标，不能补编码器盲区。

3. 触发条件

该问题更容易在以下场景出现：

- 1. CPU2 软件复位、看门狗复位、调试重启，TMC5130 和电机 24V 未同步掉电。
- 2. 运动距离较长、速度较高，重启盲区内编码轮角度变化超过半圈。
- 3. `powerOnDefaultCommand` 非 `CMD_NONE`，上电后主循环可能继续执行默认命令。如果没有“编码器就绪”和“启动安全停机”门控，会扩大位置错误影响。
- 4. 电机处于位置模式，复位前已经写入远端 `XTARGET`；TMC5130 在 MCU 复位期间仍按旧目标运行。

4. 根因拆分

4.1 启动安全边界缺失

复位后的第一优先级应该是让电机驱动处于确定的安全态，但当前 `MX_GPIO_Init()` 直接把 `DRV_ENN` 写成使能态，`App_Init()` 又没有先执行低层停机。真正清 TMC 旧目标的动作要等到 `MotorCtrl_Init()`，而它排在 1 秒延时、参数、编码器、HART、称重、Modbus、AD5421 之后。

4.2 编码器启动不等于编码器就绪

`Initialize_Encoder()` 从 FRAM 恢复的是上次保存值，并立即用这个旧值刷新 `sensor_position`。`Start_Encoder_Collection_TIM()` 只启动 TIM1；有效角度要等 TIM1 中断触发 SSI DMA，且 DMA 回调成功解析后才会调用 `Update_Encoder_Count()`。

如果电机在这个窗口内继续运动，软件没有实时增量。第一帧到来时只能把当前角度和旧 `prev_angle` 做一次最短路径差值，跨过多半圈或多圈后必然丢失圈数。

4.3 软件运动状态不能覆盖硬件残留运动

新增的 `motion_command_active` 是显示和后台轮询的门控，能避免没有新命令时把 `motor_state` 推成上/下行。但复位后该标志会回到 false，TMC5130 的 `RAMPMODE/XTARGET/VMAX` 不会随这个 RAM 标志自动清除。硬件侧旧运动必须通过 ENN 或 TMC5130 寄存器写入处理。

5. 推荐解决方案

阶段一：启动即禁用驱动输出

目标：CPU2 一恢复 GPIO 控制权，就阻止 TMC5130 旧运动继续带动电机。

建议修改：

1. 将 `MX_GPIO_Init()` 中 `DRV_ENN` 初始电平改为 `GPIO_PIN_SET`，与 `stpr_disableDriver()` 的禁用语义一致。
2. 将 `MOTOR_SPI2_CS` 初始电平改为 `GPIO_PIN_SET`，保证 TMC5130 SPI 片选空闲为高电平。
3. 如硬件允许，在 `DRV_ENN` 增加默认上拉或复位态禁用设计，避免 MCU 复位到 GPIO 配置完成之前 ENN 漂浮或被拉低。

风险：

- 禁用 ENN 会取消电机保持电流。如果机械结构不能自锁，需要确认重力负载、刹车或保持策略；必要时硬件上增加制动或让 TMC 供电复位链路 CPU2 同步。

阶段二：SPI2 初始化后立即清 TMC5130 旧目标

目标：在进入 `App_Init()` 的 1 秒延时和业务初始化之前，把 TMC5130 运动状态清成“无旧目标”。

建议新增一个早期安全函数，例如：

```
uint32_t MotorCtrl_BootSafeStop(void);
```

建议语义：

1. 不依赖 `s_motor_driver.initialized`。
2. 先保持 `DRV_ENN` 禁用，避免寄存器整理期间电机输出。
3. 尝试写 `VMAX=0`。
4. 尝试读取 `XACTUAL`，成功则写 `XTARGET=XACTUAL` 并切 `RAMPMODE=TMC5130_MODE_POSITION` 或 `TMC5130_MODE_HOLD`。
5. 如果 `XACTUAL` 读取失败，至少写 `RAMPMODE=TMC5130_MODE_HOLD` 和 `VMAX=0`，保持 ENN 禁用并返回 `MOTOR_TMC_COMM_ERROR` 供启动流程记录。
6. 清 `s_motor_driver.motion_command_active`、`motion_wait_active` 和 `debug_data.motor_state`。

调用位置建议放在 `main.c` 中 `MX_SPI2_Init()` 之后、`App_Init()` 之前。这样可以利用 SPI2 已初始化的条件，不必等待参数、编码器和通信模块。

阶段三：编码器首帧有效前禁止业务运动

目标：即使启动阶段发生异常，也不能在编码器未就绪时执行新运动命令。

建议修改：

1. 在 `AS5145.c` 增加编码器采集状态，例如 `s_encoder_first_valid_sample`、`s_encoder_last_ok_tick`。
2. `Process_SSI_Frame()` 成功解析有效帧并调用 `Update_Encoder_Count()` 后置位首帧有效。
3. 对外提供：

```
bool Encoder_IsReady(void);
uint32_t Encoder_WaitFirstValidSample(uint32_t timeout_ms);
```

4. `Initialize_Encoder()` 启动 TIM1 后等待首帧，建议超时时间 100ms 到 300ms。超时则设置编码器错误，并保持电机驱动禁用或阻止编码器记步模式下的运动。
5. `MotorCtrl_Init()` 或所有运动入口在编码器记步模式下调用统一门控：编码器未就绪时返回 `ENCODER_TIMEOUT`，不下发 `stpr_moveTo()` / `stpr_moveBy()`。

注意：

- 如果 `POSITION_COUNT_MODE_MOTOR` 下允许无编码器运行，也应把这类行为显式化，至少打印“编码器未就绪，使用电机记步源”的日志，避免现场误以为仍在闭环编码器口径下运行。

阶段四：调整 `App_Init()` 启动顺序

目标：启动阶段先处理安全，再做业务初始化。

建议顺序：

1. GPIO 默认禁用驱动。
2. SPI2 初始化后执行 `MotorCtrl_BootSafeStop()`。
3. 加载参数。
4. 初始化编码器并等待首帧有效。
5. 初始化其他业务模块。
6. 执行完整 `MotorCtrl_Init()`，恢复 TMC5130 配置和持久化位置。
7. 最后才允许 `powerOnDefaultCommand` 转成正式命令。

同时建议删除或后移 `App_Init()` 开头的固定 `HAL_Delay(1000)`。如果确实为了外部模块上电稳定需要等待，也应放在“驱动已禁用、TMC 旧目标已清除、编码器已启动”之后。

阶段五：异常和恢复策略

建议补充以下保护：

1. 如果 `MotorCtrl_BootSafeStop()` 失败，启动时保持 `DRV_ENN` 禁用，置电机或 TMC 通信错误，不执行上电默认命令。
2. 如果编码器首帧超时，在编码器记步模式下禁止运动；在电机记步模式下只允许受限动作，并打印明确日志。
3. `powerOnDefaultCommand` 执行前统一检查“启动安全停机完成”和“编码器/位置源就绪”。
4. 现场日志中增加启动阶段摘要：启动安全停机结果、编码器首帧结果、驱动使能状态、是否允许上电默认命令。

6. 验证计划

6.1 静态验证

1. 搜索 `DRV_ENN`，确认所有启动默认电平和 `stpr_disableDriver()` 语义一致。
2. 搜索 `MotorCtrl_Init()` 和所有运动入口，确认新增门控不会被串口调试、自动恢复、上电默认命令绕过。
3. 搜索 `Start_Encoder_Collection_TIM()`，确认启动成功和首帧有效有不同状态。
4. 运行 CPU2 构建：`cmake --build build\LTD_MAIN_CPU2`。

6.2 台架验证

1. 电机运行中按复位键或触发看门狗复位，观察 `DRV_ENN` 在 GPIO 初始化后是否保持禁用。
2. 复位后 TMC5130 是否不再继续执行旧 `XTARGET`。
3. 编码器首帧未到前，下发 `CMD_MOVE_UP` / `CMD_MOVE_DOWN` 应被拒绝或延后，不应启动电机。
4. 编码器正常连接时，上电日志应显示首帧有效，随后运动正常。
5. 编码器断开时，上电默认命令不应触发电机运动，系统应进入明确错误或受限状态。
6. 运行中复位后，`g_encoder_count`、`sensor_position` 与实际位置偏差应不再随复位盲区扩大。

6.3 回归验证

1. 零点标定、上/下行手动命令、单点测量、固定点监测、罐底测量均能正常初始化电机。
2. 电机 24V 断电再恢复的自动恢复流程仍能重新下发 TMC5130 配置。
3. 现有 `motion_command_active` 显示门控仍能避免静止时误显示上/下行。

7. 推荐实施优先级

1. 优先改 `DRV_ENN` 默认禁用和早期 `MotorCtrl_BootSafeStop()`，这是阻止旧运动继续的根修复。
2. 第二步补编码器首帧就绪门控，防止未采集状态下执行新运动。
3. 第三步收口 `powerOnDefaultCommand` 和自动恢复入口，避免绕过安全门控。
4. 最后再根据台架结果决定是否增加硬件上拉、制动或 TMC5130 复位/电源联动。